

Estudo de Casos



UNIFEI

Universidade Federal de Itajubá

IMC – Instituto de Matemática e Computação

Prof. Carlos Minoru Tamaki

Estudo de casos: UNIX

- Desenvolvido no Bell Labs no início da década de 1970
- Primeira versão em assembly por Ken Thompson para o minicomputador PDP-7
- Para o PDP-11 foi escrito em C que foi inventada por Dennis Ritchie
- Final da década de 1980, duas versões incompatíveis: UNIX Berkley e o System V
- Padrão POSIX (Portable Operational System IX) pelo IEEE Standards Board → Padrão Internacional

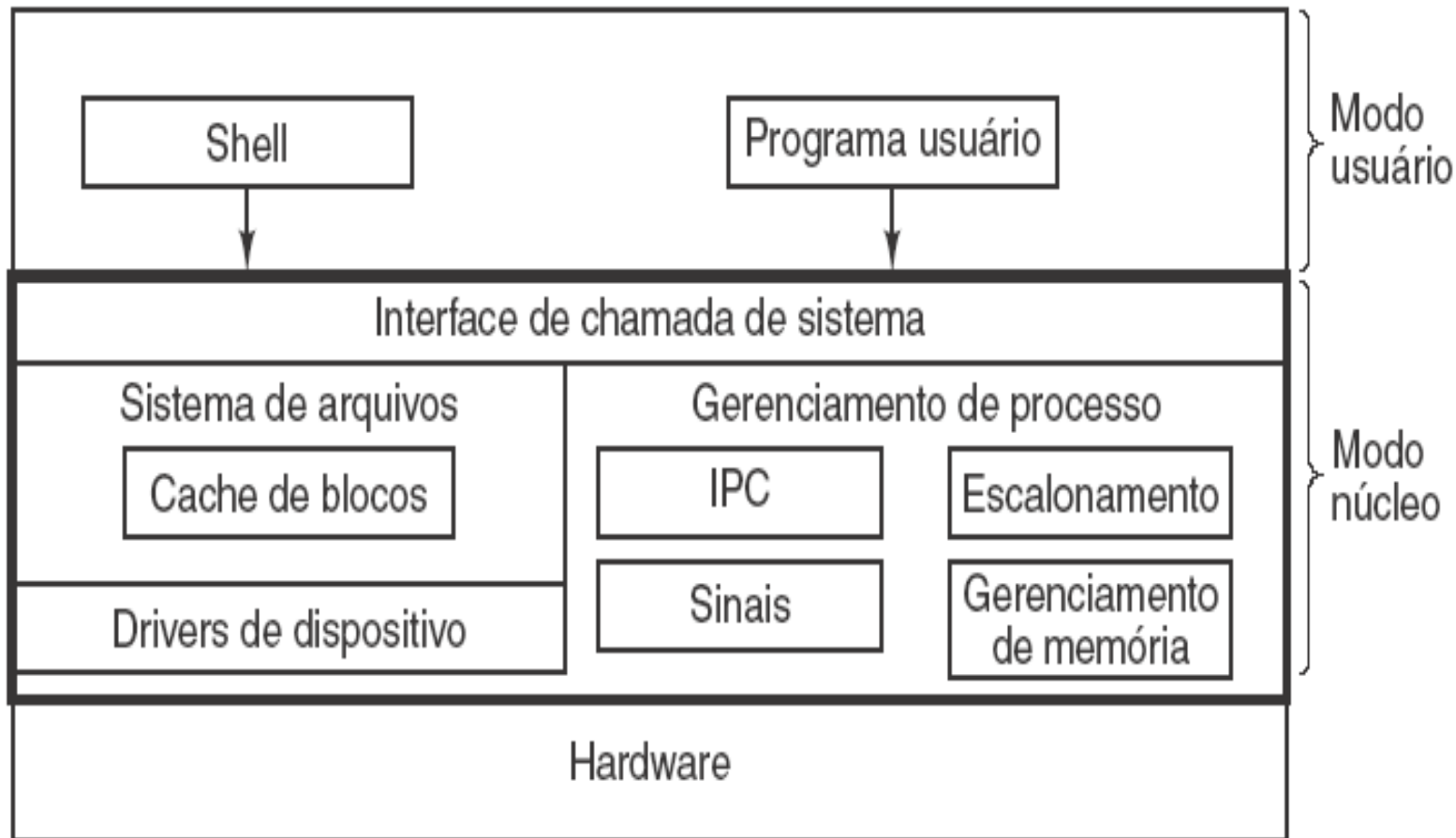
- P1003.1 → define chamadas ao sistema, em torno de 60 chamadas de sistema, como ler e escrever em arquivos, criar novos processos e outros.
 - P1003.2 → define programas utilitários básicos E outros que devem ser respeitados para pertencer ao padrão **POSIX**
- UltraSPARC III possui um sistema operacional baseado no **System V** denominado **Solaris**
- Conceito de soquete → extremidade final de uma conexão
- Usa o protocolo **TCP/IP**

Divisão aproximada de chamadas de sistema do UNIX

Categoria	Alguns exemplos
Gerenciamento de arquivo	Abrir, ler, escrever, fechar e bloquear arquivos
Gerenciamento de diretório	Criar e apagar diretórios; mover arquivos de um lado para outro
Gerenciamento de processo	Gerar, encerrar, monitorar e sinalizar processos
Gerenciamento de memória	Compartilhar memória entre processos; proteger páginas
Obter/ajustar parâmetros	Obter ID de usuário, grupo, processo; ajustar prioridade
Datas e horários	Ajustar horários de acesso a arquivos; usar temporizador de intervalo; execução de perfil
Rede	Estabelecer/aceitar conexão; enviar/receber mensagens
Diversos	Habilitar contabilidade; manipular cotas de disco; reiniciar o sistema

Tanenbaum, A.S., Austin, T.; Organização Estruturada de Computadores; São Paulo; Pearson; 6ª Ed, 2013

Estrutura de um sistema UNIX típico



Tanenbaum, A.S., Austin, T.; Organização Estruturada de Computadores; São Paulo; Pearson; 6ª Ed, 2013

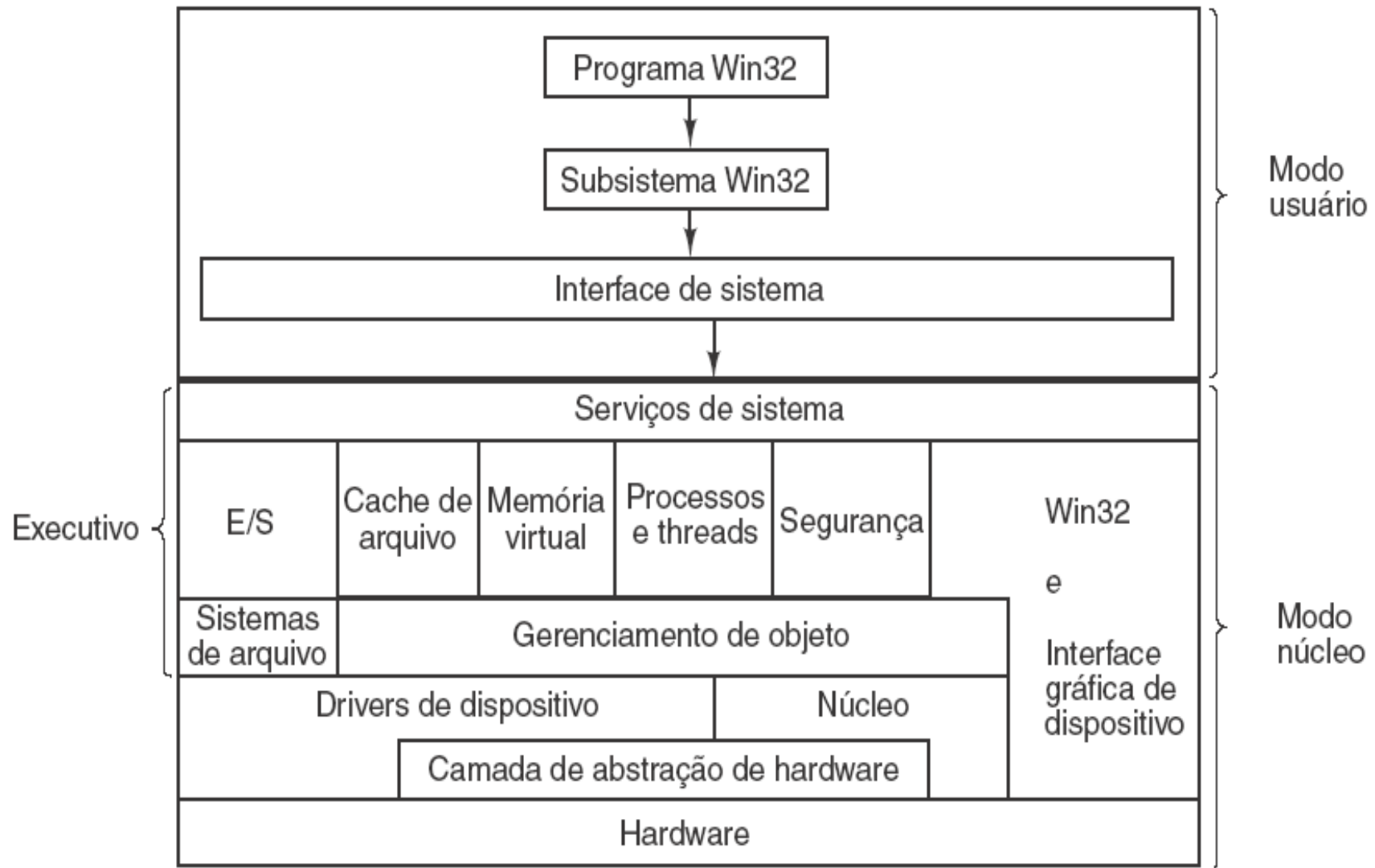
- Interfaces gráficas:
 - MIT → X Windows
- GUI – Graphical User Interface → Motif
- Operam como programas do usuário, fora do núcleo do sistema

Estudo de casos: Windows

- Início do IBM PC – com MS-DOS 1.0 em 1981
- MS-DOS → Windows 1, 2 e 3 somente interface gráfica
- 1995 – Windows 95 ainda com MS-DOS 7.0 alguns códigos de 32 bits
- 1998 – Windows 98 – MS-DOS 7.1
- Windows ME – Millennium Edition - “Windows 98 melhorado”
- Windows NT – Windows New Technology → totalmente 32 bits

- Windows 2000 – evolução do Windows NT
- Windows XP → Windows 2000 com algumas melhorias, verdadeiro sistema 32 bits com multiprogramação
- Quase todos os módulos são escritos em C/C++/C#, parte da interface gráfica escrita em C++ e uma pequena parte da camada mais baixa escrita em assembly
- Windows Vista – 2006
- Windows 7 – 2009
- Windows 8 – 2012
- Windows 10 – 2015
- Windows 11 – 2021

Estrutura do Windows

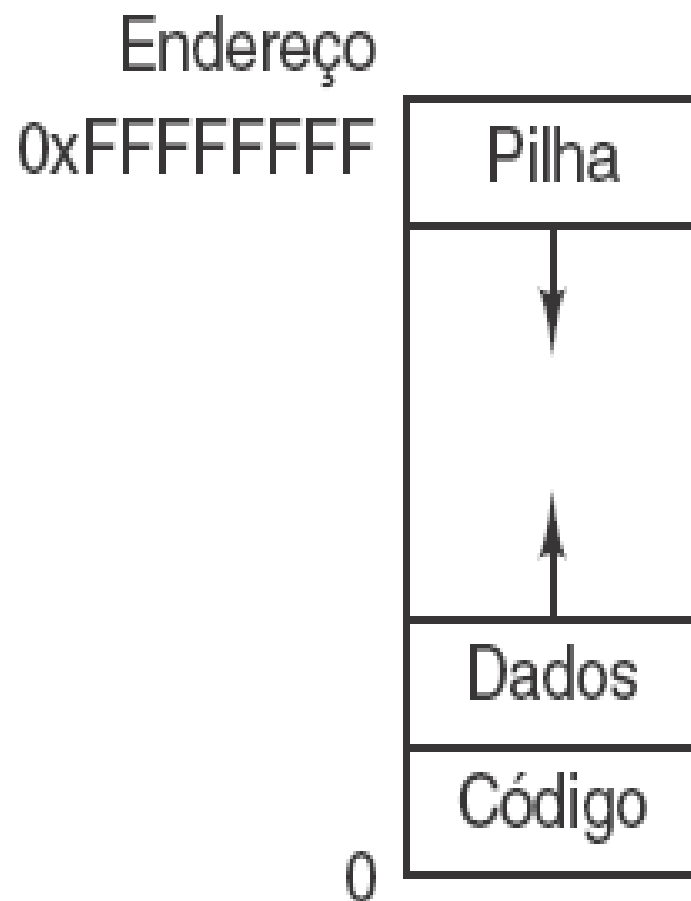


Tanenbaum, A.S., Austin, T.; Organização Estruturada de Computadores; São Paulo; Pearson; 6ª Ed, 2013

- Subsistema Win32 → módulos de biblioteca – DLL que fazem as chamadas ao sistema
- Win32 API – Application Programming Interface

Memória virtual do UNIX

Espaço de endereço de um único processo



Tanenbaum, A.S., Austin, T.; Organização Estruturada de Computadores;
São Paulo; Pearson; 6ª Ed, 2013

Memória virtual do Windows

- Processo usuário tem espaço virtual próprio de 32bits → 4GB de espaço de endereço virtual
- 2 GB baixo são disponíveis para código e dados do processo
- 2 GB alto permitem acesso limitado à memória do núcleo
- Em versões Server a divisão é diferente 3 GB para usuário e 1 GB para núcleo
- O espaço é paginado por demanda com páginas e 4 KB no Core i7
- Na versão 64bits é semelhante, porém o espaço de código e dados são nos 8 TB inferiores

Principais chamadas Windows API para gerenciar memória virtual

Função API	Significado
VirtualAlloc	Reserva ou compromete uma região
VirtualFree	Libera ou descompromete uma região
VirtualProtect	Altera a proteção ler/escrever/executar em uma região
VirtualQuery	Consulta o estado de uma região
VirtualLock	Transforma uma região da memória em residente, isto é, desabilita paginação para ela
VirtualUnlock	Torna uma região paginável do modo usual
CreateFileMapping	Cria um objeto de mapeamento de arquivo e lhe designa (opcionalmente) um nome
MapViewOfFile	Mapeia (parte de) um arquivo para o espaço de endereço
UnmapViewOfFile	Remove um arquivo mapeado do espaço de endereço
OpenFileMapping	Abre um objeto de mapeamento de arquivo previamente criado

E/S virtual em UNIX

Principais chamadas de sistema de arquivos UNIX

Chamada de sistema	Significado
<code>creat(name, mode)</code>	Cria um arquivo; <i>mode</i> especifica o modo de proteção
<code>unlink(name)</code>	Apaga um arquivo (admitindo que há só um link para ele)
<code>open(name, mode)</code>	Abre ou cria um arquivo e retorna um descritor de arquivo
<code>close(fd)</code>	Fecha um arquivo
<code>read(fd, buffer, count)</code>	Lê <i>count</i> bytes para o buffer
<code>write(fd, buffer, count)</code>	Escreve <i>count</i> bytes do buffer
<code>lseek(fd, offset, w)</code>	Move o ponteiro de arquivo como requerido por <i>offset</i> e <i>w</i>
<code>stat(name, buffer)</code>	Retorna informação sobre um arquivo
<code>chmod(name, mode)</code>	Altera o modo de proteção de um arquivo
<code>fcntl(fd, cmd, ...)</code>	Realiza várias operações de controle tais como bloquear (parte de) um arquivo

Tanenbaum, A.S., Austin, T.; Organização Estruturada de Computadores;
São Paulo; Pearson; 6ª Ed, 2013

E/S virtual em UNIX

Fragmento de programa para copiar um arquivo usando chamadas de sistema UNIX. Esse fragmento está em C porque Java oculta as chamadas de sistema de nível baixo e nós estamos tentando expô-las.

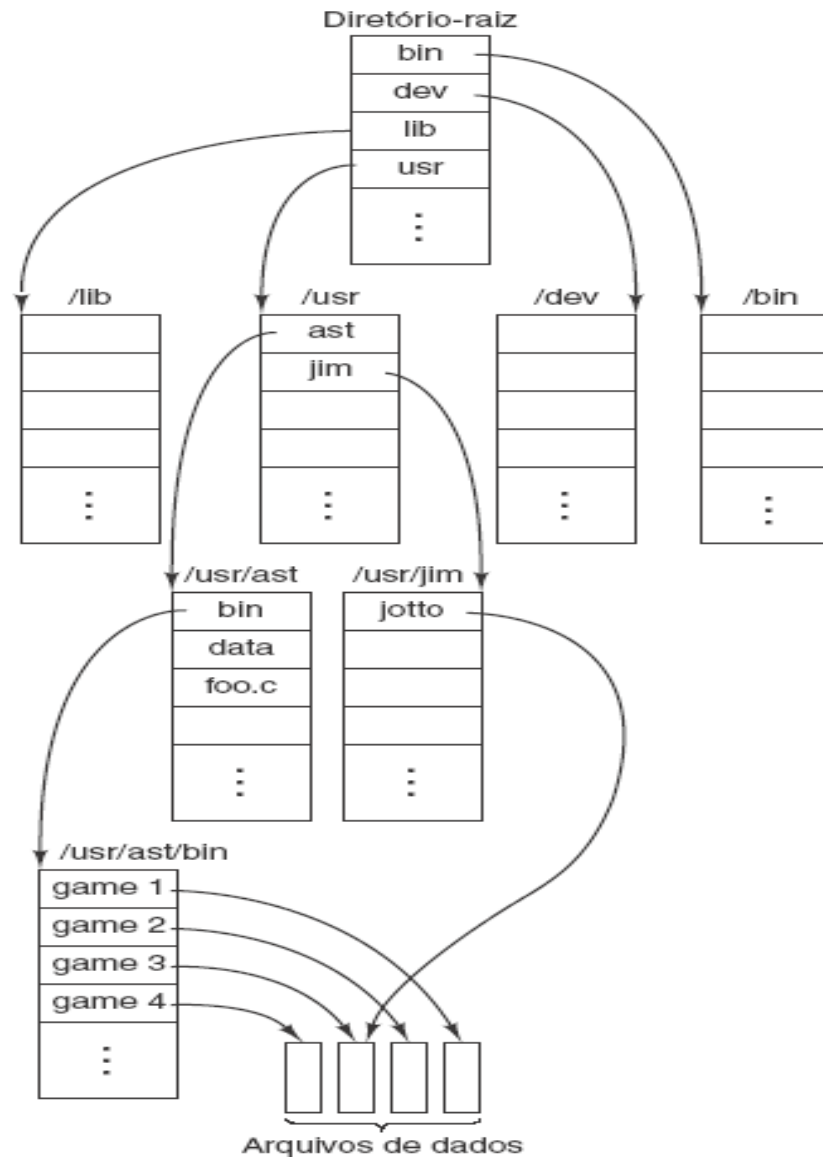
```
/* Abra os descritores de arquivo. */
infd = open("data", 0);
outfd = creat("newf", ProtectionBits);

/* Laço de cópia. */
do {
    count = read(infd, buffer, bytes);
    if (count > 0) write(outfd, buffer, count);
} while (count > 0);

/* Feche os arquivos. */
close(infd);
close(outfd);
```

E/S virtual em UNIX

Parte do sistema de diretório típico do UNIX



E/S virtual em UNIX

Principais chamadas de gerenciamento de diretórios do UNIX

Chamada de sistema	Significado
<code>mkdir(name, mode)</code>	Cria um novo diretório
<code>rmdir(name)</code>	Apaga um diretório vazio
<code>opendir(name)</code>	Abre um diretório para leitura
<code>readdir(dirpointer)</code>	Lê a próxima entrada em um diretório
<code>closedir(dirpointer)</code>	Fecha um diretório
<code>chdir(dirname)</code>	Muda diretório de trabalho para dirname
<code>link(name 1, name 2)</code>	Cria uma entrada de diretório name 2 que aponta para name 1
<code>unlink(name)</code>	Remove name de seu diretório

E/S virtual em UNIX

O formato, conteúdo e layout de um inode:

1. O tipo do arquivo, os 9 bits de proteção RWX e alguns outros bits
2. O número de links para o arquivo (o número de entrada de diretórios para eles)
3. A identidade do proprietário
4. O grupo do proprietário
5. O comprimento do arquivo em bytes
6. Treze endereços de disco
7. A última vez que o arquivo foi lido
8. A última vez que o arquivo foi escrito
9. A última vez que o inode foi alterado

E/S virtual no Windows 7

Principais funções API WIN32 para E/S de arquivos

Função API	UNIX	Significado
CreateFile	open	Cria um arquivo ou abre um arquivo existente; retorna um manipulador
DeleteFile	unlink	Destroi um arquivo existente
CloseHandle	close	Fecha um arquivo
ReadFile	read	Lê dados de um arquivo
WriteFile	write	Escreve dados para um arquivo
SetFilePointer	lseek	Ajusta o ponteiro de arquivo para um local específico no arquivo
GetFileAttributes	stat	Retorna as propriedades do arquivo
LockFile	fcntl	Bloqueia uma região do arquivo para proporcionar exclusão mútua
UnlockFile	fcntl	Desbloqueia uma região previamente bloqueada do arquivo

E/S virtual no Windows 7

Fragmento de programa
para copiar um arquivo
usando as funções da API
do Windows XP.

O fragmento está em C
porque Java oculta as
chamadas de sistema de
nível baixo e nós estamos
tentando expô-las.

```
/* Abra arquivos para entrada e saída. */
inhandle = CreateFile("data", GENERIC_READ, 0, NULL, OPEN_EXISTING, 0, NULL);
outhandle = CreateFile("newf", GENERIC_WRITE, 0, NULL, CREATE_ALWAYS,
                      FILE_ATTRIBUTE_NORMAL, NULL);

/* Copie o arquivo. */
do {
    s = ReadFile(inhandle, buffer, BUF_SIZE, &count, NULL);
    if (s > 0 && count > 0) WriteFile(outhandle, buffer, count, &ocnt, NULL);
} while (s > 0 && count > 0);

/* Feche os arquivos. */
CloseHandle(inhandle);
CloseHandle(outhandle);
```

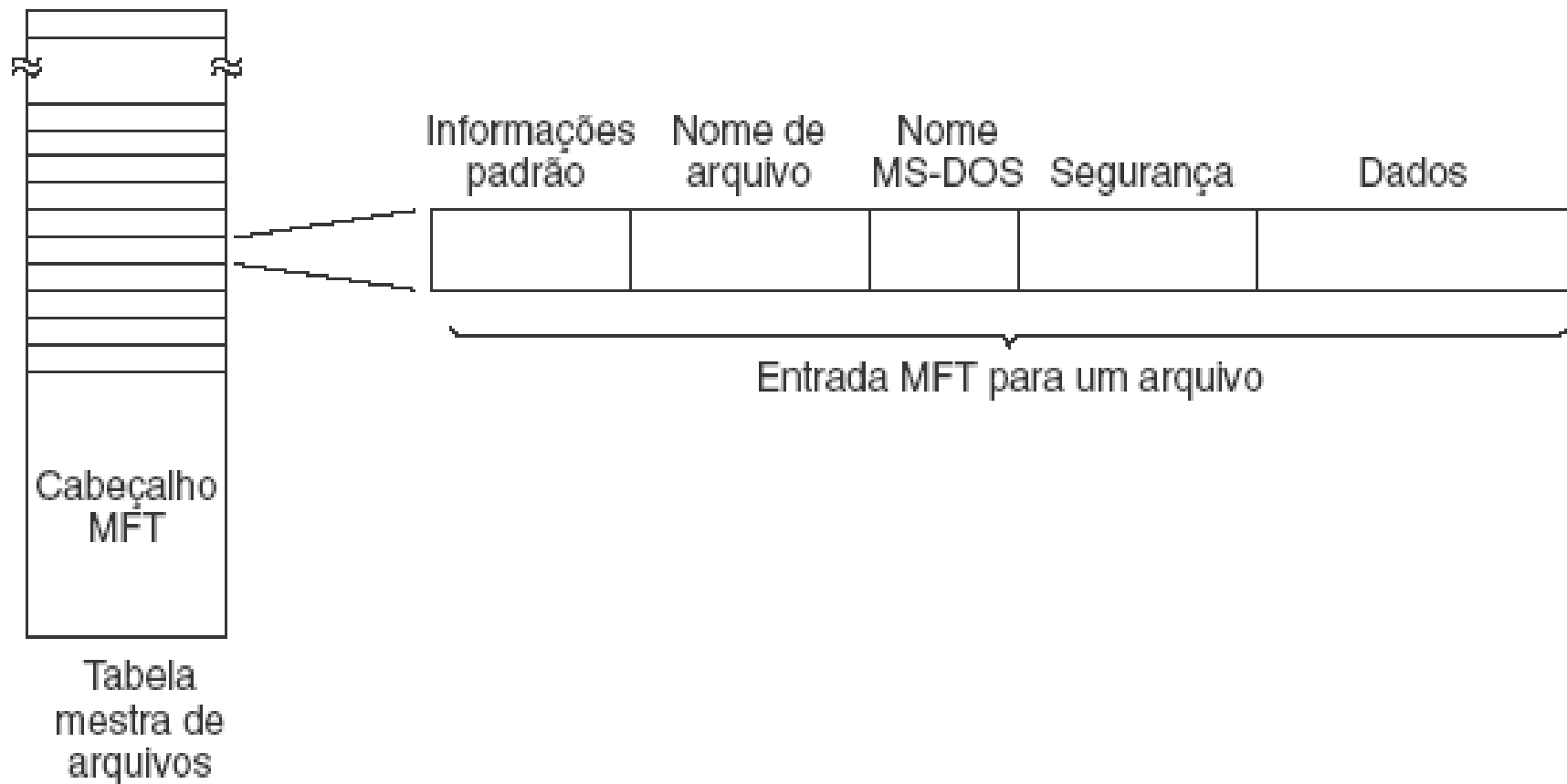
E/S virtual no Windows 7

Principais funções da API Win32 para gerenciamento de diretório.
A segunda coluna dá a mais próxima equivalente UNIX, quando ela existe.

Função API	UNIX	Significado
CreateDirectory	mkdir	Cria um novo diretório
RemoveDirectory	rmdir	Remove um diretório vazio
FindFirstFile	opendir	Inicializa e começa a ler as entradas em um diretório
FindNextFile	readdir	Lê a próxima entrada de diretório
MoveFile		Move um arquivo de um diretório para outro
SetCurrentDirectory	chdir	Muda o diretório de trabalho corrente

E/S virtual no Windows 7

Tabela mestra de arquivos do Windows 7



Gerenciamento de processo em UNIX

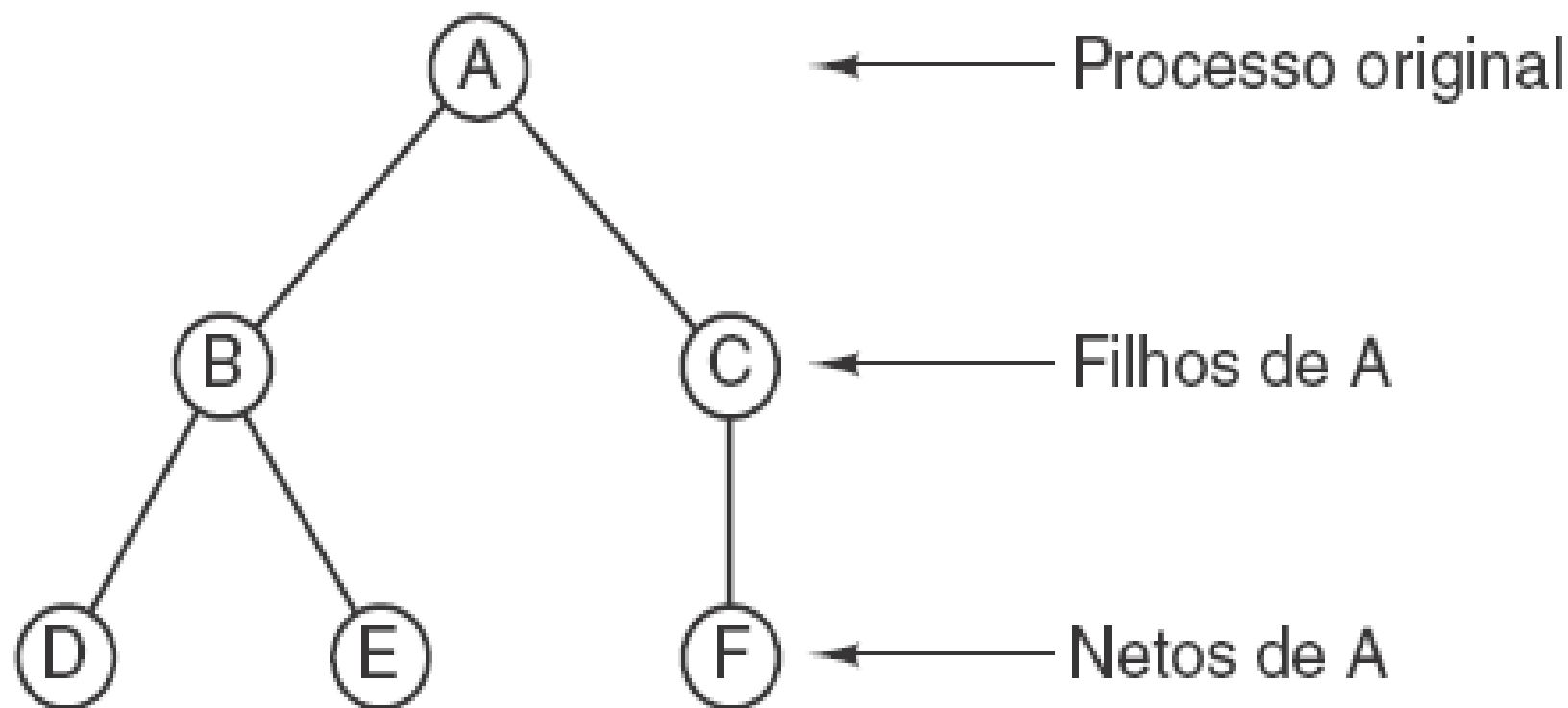
- Unix pode criar um subprocesso através da chamada de sistema ***fork***
- O processo original ***pai*** cria o processo ***filho*** que são idênticos
- Compartilham o mesmo descritor de arquivos
- Mas são independentes
- Processos podem executar ***fork*** quantas vezes quiser

Gerenciamento de processo em UNIX

- Processos Unix podem se comunicar por meio de uma estrutura denominada ***pipe*** (conexão ou tubo)
- O pipe é um buffer de memória no qual um processo pode escrever um conjunto de dados e o outro processo pode ler na ordem em que foram escritos

Gerenciamento de processo em UNIX

Árvore de processos em UNIX



Gerenciamento de processo em UNIX

- O Unix também possui a capacidade de um processo ter múltiplos ***threads***
- O padrão Unix para threads é denominado ***pthread*** e é definido por P1003.1C
- Para sincronizar as threads utiliza-se travas denominadas ***mutexes***
- Mutexes são como semáforos binários

Gerenciamento de processo em UNIX

Principais chamadas de threads POSIX

Chamada de thread	Significado
Pthread_create	Cria um novo thread no espaço de endereço do chamador
Pthread_exit	Encerra o thread que está chamando
Pthread_join	Espera que um thread encerre
Pthread_mutex_init	Cria um novo mutex
Pthread_mutex_destroy	Destrói um mutex
Pthread_mutex_lock	Bloqueia um mutex
Pthread_mutex_unlock	Desbloqueia um mutex
Pthread_cond_init	Cria uma variável de condição
Pthread_cond_destroy	Destrói uma variável de condição
Pthread_cond_wait	Espera em uma variável de condição
Pthread_cond_signal	Libera um thread que está esperando em uma variável de condição

Gerenciamento de Processos em Windows

Em termos gerais 10 parametros para o CreateProcess:

1. Um ponteiro para o nome do arquivo executável
2. A linha de comando em si (não analisada)
3. Um ponteiro para o descritor de segurança para o processo
4. Um ponteiro para o descritor de segurança para o thread inicial
5. Um bit que indica se o novo processo herda os manipuladores do criador
6. Sinalizadores diversos (por exemplo, modo erro, prioridade, depuração, consoles)
7. Um ponteiro para as cadeias ambientais
8. Um ponteiro para o nome do diretório de trabalho corrente do novo processo
9. Um ponteiro para uma estrutura que descreve a janela inicial na tela
10. Um ponteiro para uma estrutura que retorna 18 valores para o chamador

Gerenciamento de Processos em Windows

- Não há nenhum tipo de hierarquia pai-filho
- Existe um forte controle devido aos parâmetros retornados
- Cada processo é criado com um único thread, mas o processo pode criar outros threads
- O Windows fornece **fibras**, cada thread pode ter várias fibras
- A comunicação entre processos pode ser feito de várias maneiras: **pipe, pipes nomeadas, mailslots, soquetes, chamadas remotas de procedimentos e arquivos compartilhados**

Gerenciamento de Processos em Windows

- Mailslots são características do Windows, semelhantes aos pipes, mas são de uma via só e também podem ser usadas em redes, podem enviar mensagens em broadcast
- Soquetes são como pipes, mas normalmente conectam processos em máquinas diferentes
- O Windows fornece vários mecanismos de sincronização como: semáforos, mutexes, seções críticas e eventos
- Todos os mecanismos de sincronização funcionam em threads e não em processos

Bibliografia

- Organização Estruturada de Computadores
– Andrew S. Tanenbaum, Tood Austin – 6ª
Edição 2013 Prentice Hall
- Vários sites da internet